

03 - Repositories

📅 09-09-2026 ⌚ Ongeveer 23 minuten 🗨️ Ongeveer 3467 woorden

In het vorige hoofdstuk hebben we een werkende `CRUD-API` gebouwd. De data werd toen rechtstreeks bijgehouden en beheerd binnen de **ProductController**. Hoewel dat prima werkte voor een eerste demo, is dat in een echte productie-omgeving een grote architecturale fout.

Een `Controller` heeft in de back-end wereld namelijk maar één taak : de 'digitale ober' zijn. Hij neemt het verzoek aan, controleert het vluchtig, en serveert het resultaat. De ober hoort niet zelf in de keuken te staan om de biefstuk te bakken, en hij hoort al helemaal niet achterom te lopen om de voorraadkast te herorganiseren.

Om onze code overzichtelijk, testbaar en flexibel te houden, introduceren we `Repositories` en `Interfaces`. Daarnaast introduceren we `Logging` om errors en kritische informatie te kunnen noteren.

Terugblik & Focus

In het vorige hoofdstuk hebben we een functionele `CRUD API` gebouwd rondom een `Laptop-model`. De data werd toen nog handmatig beheerd via een tijdelijke in-memory lijst die rechtstreeks binnen onze controller leefde, en we leerden communiceren via de juiste `HTTP-statuscodes`.

In dit hoofdstuk breken we die strakke koppeling (`Tight Coupling`) resoluut open. We passen de principes van `Separation of Concerns` toe door alle datalogica te verhuizen naar een `Repository Pattern`. We definiëren strikte contracten via `Interfaces` en introduceren `Dependency Injection (DI)` om onze code flexibel en testbaar te maken. Tot slot voegen we `Logging` toe om de applicatie productierijp te maken.

Het Probleem: Te strakke koppeling (Tight Coupling)

Als we de database-code rechtstreeks in de controller schrijven, ontstaat er een probleem dat we `Tight Coupling` noemen. Stel je voor dat je 50 verschillende controllers hebt die allemaal rechtstreeks `SQL-code` naar een `SQL Server` sturen. Wat gebeurt er als het bedrijf besluit over te stappen naar een `PostgreSQL-database` of een `MongoDB`? Dan moet je de code in alle 50 controllers handmatig gaan opensnijden en aanpassen. Dat is niet alleen enorm veel werk, het is ook vragen om nieuwe bugs.

De Oplossing: De Repository als data-buffer

Een `Repository` is een class die functioneert als een buffer tussen je applicatie (de `Controller`) en de databron (een lijst, een bestand, of een echte database). De controller vraagt simpelweg aan de `Repository`: "Geef mij alle laptops", en hoe die `Repository` die laptops tevoorschijn tovert, zal de controller een zorg zijn.

In dit hoofdstuk gaan we deze architectuur stap voor stap opbouwen:

- We definiëren een `Interface` (het contract).
- We bouwen een `InMemoryRepository` die data wegschrijft naar een tekstbestand (.txt of .json) op je harde schijf, zodat je data écht bewaard blijft na een herstart.
- Later vervangen we deze naadloos door een `LaptopRepository` die praat met een echte database via `Entity Framework`, zonder één regel code in de controller aan te passen.

Opkuisen project

Het is een `best practice` om regelmatig verouderde en ongebruikte code uit je project te verwijderen zodat het overzichtelijk en beperkt blijft.. Verwijder `WeatherForecast.cs`, `WeatherForecastController.cs` en `WelkomController` uit het project.

Interfaces

Voordat we een `Repository` bouwen, moeten we afspraken maken. In C# gebruiken we daarvoor een `Interface`.

Je kunt een interface het beste vergelijken met een `contract` of een menukaart. Een interface *bevat geen logica, maar definieert enkel welke methoden er aanwezig moeten zijn in de klassen die dit contract ondertekenen.*

- Maak in je project een nieuwe map aan genaamd **Repositories**.
- Voeg een nieuw bestand toe aan deze map genaamd **ILaptopRepository.cs**
- In de Interface schrijven we enkel de **header(s)** van de methoden die de implementerende klasse(n) moeten voorzien.

- De Interface houdt zich enkel bezig met de **wat**, niet met de **hoe**.
 - Enkel method headers (naam, input parameters en return type),
 - Geen body (geen code).

Voeg volgende code toe aan **ILaptopRepository.cs**

```
1  using WebAPIDemo.Models;
2
3  namespace WebAPIDemo.Repositories;
4
5  public interface ILaptopRepository
6  {
7      List<Laptop> GetAll();
8      Laptop? GetById(int id);
9      List<Laptop>? GetLaptopsByMerk(string merk);
10     Laptop Create(Laptop laptop);
11     void Update(int id, Laptop laptop);
12     void Delete(int id);
13 }
```

csharp

Info

✓ In C# is het conventie dat de naam van een **Interface** met een hoofdletter **I** van *Interface* begint.

? Het vraagteken achter Laptop? GetById(int id) geeft aan dat deze methode **NULL** kan returnen.

Waarom een Interface?

Als de Controller straks praat met een interface (bijvoorbeeld ILaptopRepository), maakt het de controller niet uit welke specifieke repository erachter hangt. Zolang de klasse zich maar aan de afspraken van de interface houdt.

Info

 **Analogie: Brood in ruil voor geld** Je kunt een interface het best beschouwen als een abstract, uitwisselbaar contract. Denk aan een bakker: jij geeft geld, de bakker geeft brood. Jij geeft er als klant niet om hoe vroeg de bakker moest opstaan om het deeg te kneden, en de bakker geeft er niet om hoe jij aan je geld bent gekomen. Het contract is simpel: *geld in ruil voor brood*. Omdat dit contract universeel is, kun je ermee naar eender welke bakker ter wereld stappen.

Binnen onze API werkt een contract exact hetzelfde. De Controller is de klant die roept: "*Ik geef jou een Id, jij geeft mij een product!*" Hoe de Repository dat product tevoorschijn tovert — of die nu uit een tijdelijke lijst, een JSON-bestand, of een gigantische SQL-database komt — zal de controller een zorg zijn. Zolang de repository zich maar aan het contract houdt.

Implementerende Repository schrijven

Zoals we eerder hebben benoemd, is de **enige verantwoordelijkheid** van een Controller het afhandelen van *inkomende HTTP-requests en het terugsturen van een passend antwoord*. Je kan het vergelijken met een receptionist aan een balie: deze ontvangt de vraag van een klant, delegeert het eigenlijke werk naar de juiste afdeling (de "motor" van het bedrijf), en overhandigt het eindresultaat aan de klant zodra het klaar is. De receptionist gaat echter niet zélf het magazijn in om producten te zoeken of te monteren.

Momenteel doet onze Controller nog te veel: hij begroet niet alleen de bezoeker, maar beheert ook zélf de in-memory lijst en voert alle zoek- en bewerkingsopdrachten uit. Dit schendt het principe van *Separation of Concerns* (het scheiden van verantwoordelijkheden, SOC). Laten we de code opruimen door alle specifieke *database functionaliteit* te verhuizen naar een aparte *Repository*.

Om te beginnen zullen we blijven gebruik maken van onze in-memory lijst. Maak een nieuw bestand aan in de **Repositories** folder. Zorg ervoor dat je klasse overerft van `ILaptopRepository` en laat je code-editor de benodigde methodes (method stubs) genereren.

Info

De Interface dicteert enkel *wat* onze implementerende klasse moet kunnen (welke methodes er moeten zijn). De manier *hoe* dat doel bereikt wordt, bijvoorbeeld met een tijdelijke lijst, een tekstbestand of een echte SQL-database, is volledig de keuze van de implementerende klasse.

Verplaats nu alle data-logica uit de Controller naar **InMemoryRepository.cs**, zodat het bestand er als volgt uitziet:

```
1      using WebAPIDemo.Models;                                     csharp
2
3      namespace WebAPIDemo.Repositories;
4
5      public class InMemoryRepository : ILaptopRepository
6      {
7          // Onze tijdelijke "database" in het geheugen
8          private List<Laptop> laptops = new List<Laptop>
9          {
10              new Laptop { Id = 1, Merk = "Dell", Processor = "Intel i7",
11              RamInGB = 16, Prijs = 1200.00, GPU = "Iris Xe" },
12              new Laptop { Id = 2, Merk = "Apple", Processor = "M3 Pro",
13              RamInGB = 18, Prijs = 2500.00, GPU = "Apple GPU" },
14              new Laptop { Id = 3, Merk = "Lenovo", Processor = "AMD Ryzen 7",
15              RamInGB = 32, Prijs = 1400.00, GPU = "RTX 4060" }
16          };
17
18          public List<Laptop> GetAll()
19          {
20              return laptops;
21          }
22
23          public Laptop? GetById(int id)
24          {
25              // Zoek de laptop in de lijst die de gevraagde Id heeft
26              return laptops.FirstOrDefault(x => x.Id == id);
27          }
28
29
30          public List<Laptop>? GetLaptopsByMerk(string merk)
31          {
32              //Zoek laptops waar een bepaalde conditie true is
33              return laptops
34                  .Where(x => x.Merk.ToLower() == merk.ToLower())
35                  .ToList();
36          }
37
38          public Laptop Create(Laptop nieuweLaptop)
39          {
40              // Omdat we geen database hebben die automatisch ID's genereert,
```

```
40     zoeken we zelf de hoogste ID en tellen we er 1 bij op.
41     int nieuweId = laptops.Max(x => x.Id) + 1;
42     nieuweLaptop.Id = nieuweId;
43
44     // Voeg de nieuwe laptop toe aan onze in-memory lijst
45     laptops.Add(nieuweLaptop);
46     return nieuweLaptop;
47 }
48
49 public void Update(int id, Laptop bijgewerkteLaptop)
50 {
51     // Zoek de laptop die momenteel in de lijst staat
52     var bestaandeLaptop = laptops.FirstOrDefault(x => x.Id == id);
53
54     if (bestaandeLaptop != null)
55     {
56         // Overschrijf de eigenschappen met de nieuwe data uit de
57 body
58         bestaandeLaptop.Merk = bijgewerkteLaptop.Merk;
59         bestaandeLaptop.Processor = bijgewerkteLaptop.Processor;
60         bestaandeLaptop.RamInGB = bijgewerkteLaptop.RamInGB;
61         bestaandeLaptop.Prijs = bijgewerkteLaptop.Prijs;
62         bestaandeLaptop.GPU = bijgewerkteLaptop.GPU;
63     }
64 }
65
66 public void Delete(int id)
67 {
68     // Zoek de laptop in onze lijst
69     var laptop = laptops.FirstOrDefault(x => x.Id == id);
70
71     if (laptop != null)
72     {
73         // Verwijder het object uit de lijst
74         laptops.Remove(laptop);
75     }
76 }
77 }
```

Nadat al het databeheer is verplaatst naar de Repository, kunnen we de Controller gaan herschrijven (refactoren). De Controller krijgt nu rust: hij hoeft enkel nog requests in ontvangst te nemen en een gepast HTTP-antwoord (zoals 200 OK of 404 Not Found) te re-

turneren. Telkens wanneer er data nodig is, delegeert hij deze vraag simpelweg naar een `ILaptopRepository` .

Om dit te realiseren, definiëren we bovenaan in de Controller een nieuwe **private** variabele van het type `ILaptopRepository` . We noemen een dergelijke variabele een `Dependency` (een afhankelijkheid). Onze Controller is immers afhankelijk van deze component om zijn werk te kunnen uitvoeren. Noem deze variabele `_laptopRepository` .

Vervolgens bepalen we in de **constructor van de Controller** welke concrete klasse we aan deze interface koppelen. *Dit is de enige regel in de volledige Controller die weet welk specifiek type repository er achter de schermen wordt gebruikt.* Dit zorgt voor zogenaamde "losse koppeling" (*Loose Coupling*). Omdat de rest van de code enkel communiceert met de interface, kunnen we in de toekomst moeiteloos overschakelen naar een échte database door simpelweg deze ene regel in de constructor te wijzigen.

```
1  [ApiController]
2  [Route("api/[controller]")] // Route wordt: api/laptops
3  public class ProductController : ControllerBase
4  {
5      private ILaptopRepository _laptopRepository;
6
7      public ProductController()
8      {
9          _laptopRepository = new InMemoryRepository();
10     }
11
12     // ...
13 }
```

csharp

Tips

In C# is het conventie dat de naam van een **Dependency** met een underscore `_` begint.

Pas nu de rest van de Controller aan zodat deze **nooit rechtstreeks** met de database communiceert.

```
1  using Microsoft.AspNetCore.Mvc;
2  using WebAPIDemo.Models;
3  using WebAPIDemo.Repositories;
4
5  namespace WebAPIDemo.Controllers;
```

csharp

```
6
7 [ApiController]
8 [Route("api/[controller]")] // Route wordt: api/laptops
9 public class ProductController : ControllerBase
10 {
11     private ILaptopRepository _laptopRepository;
12
13     public ProductController()
14     {
15         _laptopRepository = new InMemoryRepository();
16     }
17
18     // 1. GET: Alle laptops ophalen
19     // Route: GET api/laptops
20     [HttpGet]
21     public ActionResult<List<Laptop>> GetAlleLaptops()
22     {
23         // We sturen een 200 OK status terug samen met de lijst
24         return Ok(_laptopRepository.GetAll());
25     }
26
27     // 2. GET: Eén specifieke laptop op basis van Id
28     // Route: GET api/laptops/1
29     [HttpGet("{id}")]
30     public ActionResult<Laptop> GetLaptopById(int id)
31     {
32         Laptop laptop = _laptopRepository.GetById(id);
33
34         // Als de laptop niet bestaat, stuur dan een 404 terug
35         if (laptop == null)
36         {
37             return NotFound($"Helaas, we konden geen laptop vinden met Id
38 {id}.");
39         }
40
41         return Ok(laptop);
42     }
43
44     [HttpGet("merk/{merk}")]
45     public ActionResult<List<Laptop>> GetLaptopsByMerk(string merk)
46     {
47         List<Laptop>? laptops = _laptopRepository.GetLaptopsByMerk(merk);
48     }
```

```
49         if (laptops == null || laptops.Count == 0)
50         {
51             return NotFound($"Helaas, we konden geen laptops vinden met
52 merk {merk}.");
53         }
54
55         return Ok(laptops);
56     }
57
58     // 3. POST: Een nieuwe laptop toevoegen
59     // Route: POST api/laptops
60     [HttpPost]
61     public ActionResult<Laptop> CreateLaptop(Laptop nieuweLaptop)
62     {
63         Laptop aangemaakteLaptop =
64         _laptopRepository.Create(nieuweLaptop);
65
66         // We retourneren een 201 Created statuscode.
67         // Dit vertelt de client dat het object succesvol is aangemaakt
68         én waar het te vinden is.
69         return CreatedAtAction(nameof(GetLaptopById), new { id =
70 aangemaakteLaptop.Id }, aangemaakteLaptop);
71     }
72
73     // 4. PUT: Een bestaande laptop volledig bijwerken
74     // Route: PUT api/laptops/1
75     [HttpPut("{id}")]
76     public ActionResult UpdateLaptop(int id, Laptop bijgewerkteLaptop)
77     {
78         Laptop bestaandeLaptop = _laptopRepository.GetById(id);
79
80         // 2. Als de laptop niet bestaat, sturen we een 404 Not Found
81         if (bestaandeLaptop == null)
82         {
83             return NotFound($"Kan geen laptop bijwerken met Id {id},
84 omdat deze niet bestaat.");
85         }
86
87         _laptopRepository.Update(id, bijgewerkteLaptop);
88
89         // 4. Update succesvol! We sturen een 204 No Content terug.
90         return NoContent();
91     }
```

```
92
93
94     // 5. DELETE: Een laptop verwijderen
95     // Route: DELETE api/laptops/1
96     [HttpDelete("{id}")]
97     public ActionResult DeleteLaptop(int id)
98     {
99         Laptop laptop = _laptopRepository.GetById(id);
100
101         // 2. Bestaat hij niet Dan kunnen we hem ook niet verwijderen ->
102         404!
103         if (laptop == null)
104         {
105             return NotFound($"Kan laptop met Id {id} niet verwijderen,
106 omdat deze niet is gevonden.");
107         }
108
109         _laptopRepository.Delete(id);
110
111         // 4. Succesvol verwijderd -> We sturen een 204 No Content terug
112         return NoContent();
113     }
114 }
```

Notitie

Merk op dat we in deze Controller nu 3 GET endpoints hebben. Dat is geen probleem zolang dat er voor elk endpoint **een unieke route wordt voorzien**. De naam van de methodes maakt niet uit aangezien een client deze nooit te zien krijgt. Door een string naam toe te voegen aan de HTTP Verb kan je een specifieke route voorzien. De combinatie van HTTPVerb + Route moet steeds uniek zijn , anders krijgen we een Exception.

- GetAll -> GET [url]/laptop
- GetById -> GET [url]/laptop/1
- GetLaptopsByMerk -> Get [url]/laptop/merk/acer
- Create -> POST [url]/laptop
- Update -> PUT [url]/laptop/1
- Delete -> DELETE [url]/laptop/1

Run je applicatie en test je endpoints. Als alles goed gegaan is zou je applicatie nog steeds moeten werken als voorheen.

Info

Refactoring We hebben de data-code verplaatst van onze controller naar een specifieke repository-klasse. We hebben de werking van onze API voor de eindgebruiker niet veranderd, maar de interne structuur is nu vele malen strakker en overzichtelijker.

Dit proces noemen we *Refactoring : het opschonen, herstructureren en verbeteren van bestaande code, zonder dat het gedrag van de applicatie verandert*. Een goede developer refactort zijn code regelmatig om te voorkomen dat een project verandert in onleesbare spaghetti-code .



Dependency Injection

Hoewel onze Controller nu een stuk minder werk te doen heeft, heeft deze eigenlijk nog steeds één taak te veel. *Naast het afhandelen van requests, beslist de Controller momenteel zélf welk specifiek type Dependency (de Repository) hij aanmaakt*. Dit zou niet mogen.

Laten we de vergelijking maken met een auto. Er is absoluut niets mis met het hebben van *dependencies*. Een auto heeft een motor en een set banden nodig om te kunnen rijden. Maar een auto gaat niet zélf zijn motor installeren of zijn eigen banden vervangen. Deze onderdelen worden in een fabriek gebouwd en vervolgens in de auto geplaatst. We *injec-teren* de motor in de daartoe voorziene ruimte om een werkende auto te bekomen, zonder dat de auto zelf hoeft te weten hoe die motor in elkaar is gezet.

Om dit in onze code op te lossen, maken we gebruik van **Dependency Injection** (een patroon dat voortkomt uit het *Dependency Inversion* principe). Het kernidee is simpel: *het aanmaken van een dependency mag geen verantwoordelijkheid zijn van de klasse die deze gebruikt*.

In plaats van zelf een object aan te maken (en dus het `new` trefwoord te gebruiken), laten we de applicatie de kant-en-klare functionaliteit van buitenaf in onze Controller *injecteren*. De verantwoordelijkheid voor het creëren van de repository wordt letterlijk omgedraaid en buiten de klasse gelegd. De Controller eist simpelweg via zijn constructor een `ILaptopRepository` op en gaat er blindelings vanuit dat het systeem hem een werkend exemplaar aanlevert.

Tips

🤔 Wat is de makkelijkste manier om iets te weten te komen? Het vragen .

In plaats van zelf , al dan niet complexe, creatielogica te schrijven zullen onze klassen vanaf nu *vragen* om kant-en-klare functionaliteit in de **constructor**. Lees ook [deze blogpost](#) over waarom dependencies 'nemen' geen goed idee is.

```
1  [ApiController]
2  [Route("api/[controller]")] // Route wordt: api/laptops
3  public class ProductController : ControllerBase
4  {
5      private ILaptopRepository _laptopRepository;
6
7      public ProductController(ILaptopRepository laptopRepository)
8      {
9          _laptopRepository = laptopRepository;
10     }
11
12     // ...
13 }
```

csharp

Het vragen van dependencies is allemaal goed en wel maar uiteindelijk moet *iemand* die objecten natuurlijk wel aanmaken. We kunnen niet zomaar ergens om vragen en verwachten dat het op magische wijze uit de lucht komt vallen.

Gelukkig heeft ASP.NET Core een ingebouwde motor (de zogenaamde Dependency Injection Container) die dit voor ons afhandelt. Het enige wat wij hoeven te doen, is onze classes vooraf registreren in `Program.cs` . Voor onze repository gebruiken we hiervoor de methode `AddScoped<Interface, Implementatie>()` .

Voeg de volgende regel toe aan **Program.cs** (zorg dat dit gebeurt vóór de `builder.Build()` lijn):

```
1 // Registreer de dependency in de DI container  
2 builder.Services.AddScoped<ILaptopRepository, InMemoryRepository>();
```

csharp

Wat gebeurt hier precies? We geven links de Interface op en rechts de concrete implementatie. We vertellen ASP.NET Core in feite: "Telkens wanneer een klasse (zoals onze Controller) vraagt om de `ILaptopRepository` interface, maak dan een nieuw exemplaar van de `InMemoryRepository` klasse aan en geef dat mee." De applicatie vult zo automatisch alle 'placeholders' in met de juiste functionaliteit.

Info

Er zijn verschillende manieren om Dependency Injection op te zetten, bijvoorbeeld door zelf een klasse te schrijven die alle configuratie regelt, of door factories te gebruiken. Zolang een klasse maar niet zélf (new) zijn eigen dependencies aanmaakt, respecteer je het principe.

Voor deze opleiding houden we het simpel en gebruiken we standaard de ingebouwde methode via `Program.cs` en `AddScoped()`.

Logging gebruiken

Veel applicaties gebruiken logging (*het wegschrijven van systeeminformatie*) om te kunnen bijhouden wat er op een bepaald tijdstip in de applicatie is gebeurd. Dit is handig om bij te houden hoe en wanneer een applicatie gebruikt wordt, en onmisbaar om na te gaan hoe bepaalde errors in de applicatie zijn geslopen.

In ASP.NET maken we hiervoor gebruik van de ingebouwde `ILogger` interface. Omdat dit óók een externe afhankelijkheid is, gaan we deze **net als onze repository injecteren via de constructor**.

👉 **Tip:** Beschouw Dependencies als de ingrediënten van een recept en zet ze overzichtelijk in een lijstje (als private fields) bovenaan in de klasse. Op die manier zie je onmiddellijk welke afhankelijkheden je klasse heeft en of de klasse stiekem niet te veel verantwoorde-lijkheden draagt. Net zoals bij een recept wil je vooraf weten of je alles in huis hebt!

Er zijn verschillende niveaus van logging die je kunt gebruiken om de ernst van het bericht aan te geven:

- **LogTrace:** Gedetailleerde log, voornamelijk voor debugging.

- **LogDebug**: Informatie voor debugging.
- **LogInformation**: Informatie over normale werking.
- **LogWarning**: Warnings of mogelijke problemen.
- **LogError**: Fouten die zijn opgetreden.
- **LogCritical**: Kritieke fouten die onmiddellijke aandacht vereisen.



Gebruik A.I. om bij elke methode in de Controller minstens 1 lijn logging te voorzien

```
1  using Microsoft.AspNetCore.Mvc;
2  using WebAPIDemo.Models;
3  using WebAPIDemo.Repositories;
4  // Zorg ervoor dat je Microsoft.Extensions.Logging hebt via using
5
6  namespace WebAPIDemo.Controllers;
7
8  [ApiController]
9  [Route("api/[controller]")]
10 public class ProductController : ControllerBase
11 {
12     // Onze "ingrediënten" (Dependencies)
13     private readonly ILaptopRepository _laptopRepository;
14     private readonly ILogger<ProductController> _logger;
15
16     // Vraag om en injecteer Dependencies
17     public ProductController(ILaptopRepository laptopRepository,
18     ILogger<ProductController> logger)
19     {
20         _laptopRepository = laptopRepository;
21         _logger = logger;
22     }
23
24     [HttpGet]
25     public ActionResult<List<Laptop>> GetAllLaptops()
26     {
27         _logger.LogInformation("GET request ontvangen voor alle
28 laptops.");
29         return Ok(_laptopRepository.GetAll());
30     }
31
32     [HttpGet("{id}")]
33     public ActionResult<Laptop> GetLaptopById(int id)
34     {
```

```
35         _logger.LogInformation($"GET request voor laptop met ID: {id}");
36
37         var laptop = _laptopRepository.GetById(id);
38
39         if (laptop == null)
40         {
41             _logger.LogWarning($"Laptop met ID {id} werd niet
42 gevonden.");
43             return NotFound($"Helaas, we konden geen laptop vinden met Id
44 {id}.");
45         }
46
47         return Ok(laptop);
48     }
49
50     [HttpGet("merk/{merk}")]
51     public ActionResult<List<Laptop>> GetLaptopsByMerk(string merk)
52     {
53         _logger.LogInformation($"GET request ontvangen voor laptops met
54 merk: {merk}");
55         List<Laptop>? laptops = _laptopRepository.GetLaptopsByMerk(merk);
56
57         if (laptops == null || laptops.Count == 0)
58         {
59             _logger.LogWarning($"Geen laptops gevonden met merk:
60 {merk}");
61             return NotFound($"Helaas, we konden geen laptops vinden met
62 merk {merk}.");
63         }
64
65         return Ok(laptops);
66     }
67
68     [HttpPost]
69     public ActionResult<Laptop> CreateLaptop(Laptop nieuweLaptop)
70     {
71         _logger.LogInformation($"Aanmaken van nieuwe laptop:
72 {nieuweLaptop.Merk}");
73
74         var aangemaakteLaptop = _laptopRepository.Create(nieuweLaptop);
75
76         _logger.LogInformation($"Laptop succesvol aangemaakt met ID:
77 {aangemaakteLaptop.Id}");
```

```
78         return CreatedAtAction(nameof(GetLaptopById), new { id =
79 aangemaakteLaptop.Id }, aangemaakteLaptop);
80     }
81
82     [HttpPut("{id}")]
83     public ActionResult UpdateLaptop(int id, Laptop bijgewerkteLaptop)
84     {
85         _logger.LogInformation($"Update request voor laptop met ID:
86 {id}");
87
88         var bestaandeLaptop = _laptopRepository.GetById(id);
89
90         if (bestaandeLaptop == null)
91         {
92             _logger.LogWarning($"Update mislukt: Laptop met ID {id}
93 bestaat niet.");
94             return NotFound($"Kan geen laptop bijwerken met Id {id},
95 omdat deze niet bestaat.");
96         }
97
98         _laptopRepository.Update(id, bijgewerkteLaptop);
99
100        _logger.LogInformation($"Laptop met ID {id} is succesvol
101 bijgewerkt.");
102        return NoContent();
103    }
104
105    [HttpDelete("{id}")]
106    public ActionResult DeleteLaptop(int id)
107    {
108        _logger.LogInformation($"Delete request voor laptop met ID:
109 {id}");
110
111        var laptop = _laptopRepository.GetById(id);
112
113        if (laptop == null)
114        {
115            _logger.LogError($"Delete mislukt: Laptop met ID {id} werd
116 niet gevonden.");
117            return NotFound($"Kan laptop met Id {id} niet verwijderen,
118 omdat deze niet is gevonden.");
119        }
120    }
```

```
        _laptopRepository.Delete(id);

        _logger.LogInformation($"Laptop met ID {id} is succesvol
verwijderd.");
        return NoContent();
    }
}
```

TODO: Primary constructors toevoegen

Als je de API runt en enkele endpoints uittest, zal je de logging-messages zien op de achterliggende terminal. Uiteraard is het later heel eenvoudig om deze terminal-logger later te vervangen door een FileLogger, CloudLogger, enz. dankzij dependency inversion.

Tips

Het is niet nodig om de ILogger te registreren in Program.cs, aangezien dit een ingebouwde library is. Onze eigen dependencies moeten we wel registreren.

```
info: WebAPIDemo.Controllers.LaptopsController[0]
      GET request ontvangen voor alle laptops.
info: WebAPIDemo.Controllers.LaptopsController[0]
      GET request voor laptop met ID: 1
info: WebAPIDemo.Controllers.LaptopsController[0]
      GET request voor laptop met ID: 10
warn: WebAPIDemo.Controllers.LaptopsController[0]
      Laptop met ID 10 werd niet gevonden.
info: WebAPIDemo.Controllers.LaptopsController[0]
      Delete request voor laptop met ID: 1
info: WebAPIDemo.Controllers.LaptopsController[0]
      Laptop met ID 1 is succesvol verwijderd.
```

Samenvatting

- **Controller** : Handelt alleen inkomende HTTP-requests en uitgaande responses af.
- **Repository** : Regelt alle logica en acties rondom de data (zoeken, opslaan, bewerken, verwijderen).
- **Routes** : We kunnen oneindig veel endpoints toevoegen, zelfs met dezelfde HTTPVerb, zolang de combinatie van route + verb uniek is.

- **Interface** : fungeert als een ' contract ' in je code, bevat enkel de 'wat', niet de 'hoe'.
 - De controller vertrouwt op dit contract, maar is zich totaal onbewust van de onderliggende opslag (database, memory, etc.). Dit houdt het systeem flexibel en makkelijk te testen.(loosely coupled)
- **Refactoring** : Het herschrijven, opkuisen en optimaliseren van bestaande code zonder dat er iets aan de werking van het systeem verandert.
- **Dependency Injection** : Vermijdt het gebruik van het new sleutelwoord om afhankelijkheden aan te maken; injecteer ze via de constructor.
 - Configureer in Program .cs (bijvoorbeeld met builder.Services. AddScoped) welke concrete klasse ASP.NET Core moet aanleveren wanneer een interface wordt opgevraagd.
 - Injecteer de ingebouwde ILogger om procesinformatie, waarschuwingen en foutmeldingen bij te houden.

Laatst bijgewerkt:09-09-2026, 08:52

Bijdragers: JorenSynaeveTM